

Research Mode Implementation Plan

Web Search · Auto-Ingest · AI Agent Integration

Version 1.0 · May 2026 · Confidential

Implementation Plan

Web Search Integration · Auto-Ingest Pipeline · Safety Controls

Version 1.0 · May 2026 · core_agentic / TeamOS · Confidential

Table of Contents

1. Executive Summary	3
2. Problem Statement	3
3. Provider Decision — Tavily vs Serper	4
4. Architecture Overview	4
5. Full SSE Event Flow	5
6. Phase 1 — Tool Schemas & Implementations	6
7. Phase 2 — ResearchAgent Role & Routing	8
8. Phase 3 — Auto-Ingest Pipeline & Deduplication	10
9. Phase 4 — Safety, Quotas & Content Controls	11
10. Phase 5 — Database Models, Settings & Migration	13
11. Complete File Change Summary	14
12. Implementation Timeline	15
13. Production Checklist	15

1. Executive Summary

This document is the complete technical implementation plan for adding a web research capability to the core_agentic system powering TeamOS. Currently, the agent can only answer questions from the team's internal wiki and planning data. Teams regularly ask about competitor products, regulatory frameworks, technical standards, and market information that does not exist in the internal knowledge base.

The research mode adds three new tools (web_search, web_read_page, research_save_to_wiki), a new ResearchAgent specialist, an async ingest pipeline that automatically embeds and links discovered knowledge into the wiki graph, and a comprehensive safety layer including per-team monthly quotas, domain blocklists, content length caps, and runtime enforcement via WorkingMemory.

5	9	3	~13 hrs
Phases	Files changed	New tools	Est. dev time

2. Problem Statement

The agent's strict RAG-only design enforces a hard boundary: if the information is not in the wiki or planning data, the agent refuses to answer from general knowledge. This is correct behaviour for internal questions, but creates a frustrating dead end for external research questions. Common failure cases include:

- Competitor analysis — 'What features does Notion have that we're missing?'
- Regulatory frameworks — 'What does GDPR Article 17 require for data deletion?'
- Technology choices — 'How does Temporal compare to Celery for workflow orchestration?'
- Market data — 'What is the current market size for AI productivity tools?'
- Technical standards — 'What does ISO 27001 require for access control?'

Current agent response to external queries

"No relevant team knowledge was found in the wiki or project plans for this query. This information was not found in the team's indexed knowledge. Consider documenting this in the Wiki or ingesting more data."

3. Provider Decision — Tavily vs Serper

Tavily — RECOMMENDED	Returns pre-cleaned markdown via <code>include_raw_content=True</code> . Built-in relevance scoring per result. Native domain allow/block list parameters. \$5 per 1,000 searches. No scraping code required — content arrives summarisable immediately. The <code>/extract</code> endpoint fetches full-page content from a single URL for FALLBACK , also returning clean markdown.	Returns URLs separate scrap. Breaks on Java searches (che infrastructure, fallback strate. Tavily is unusable quality.
-----------------------------	--	---

4. Architecture Overview

The research mode plugs into the existing `universal_stream.py` routing architecture as a fourth execution path — Case D — alongside the existing Strategic Planner (A), Lightweight (B), and Operational Specialist (C) paths. All existing paths are completely untouched. The new ResearchAgent is a specialist in the multi_agent orchestrator and follows the same AgentConfig + AgentCore execution model.

Case	Trigger	Execution Path	Tools
A	STRATEGIC_PLANNER	run_planner_agent_v2 — 6-stage pipeline	planning tools, wiki
B	LIGHTWEIGHT	AgentCore, tools=[], single LLM call	none
C	WIKI / PLAN / ANALYST	AgentCore full tool loop, 10 rounds max	specialist whitelist
D (NEW)	RESEARCH	AgentCore, 6 rounds max, research tools	web_search, web_read_page, research_save,

5. Full SSE Event Flow

To ensure a fully responsive UI, the research loop emits detailed server-sent events (SSE) via `universal_stream.py`. This allows the frontend to display research step-indicators, active tickers, and toggleable log cards showing raw search queries and matching URLs.

Event	Payload / Description
<code>research_start</code>	Emitted when routing layer selects Case D. Payload contains the raw query and initial status text.
<code>research_search_call</code>	Fired immediately before hitting Tavily <code>/search</code> . Payload contains the optimized <code>search_query</code> array.
<code>research_search_results</code>	Fired when Tavily returns data. Contains a list of titles, URLs, and relevance scores to render as clickable links.
<code>research_read_call</code>	Fired when the agent decides to scrape a specific page via <code>/extract</code> . Contains target URL.
<code>research_read_complete</code>	Fired when full-text content is fetched. Contains content character count and short snippet.
<code>research_save_wiki</code>	Fired when <code>research_save_to_wiki</code> is executed. Triggers the internal document generation stream indicator.
<code>research_complete</code>	Final research sequence wrapper. Emitted right before standard message token streaming begins.

PHASE 1

Tool Schemas & Implementations

Core integration wrappers for Tavily client API

Est. time: 3.0 hours

We implement three new asynchronous Python functions inside `core_agentic/tools/research_tools.py`. These functions are strictly type-hinted and match our standardized system tool registration protocols.

TOOL CODE MOCKUP

```

import os, httpx
from typing import List, Dict, Any

async def web_search(query: str, max_results: int = 5) -> Dict[str, Any]:
    """Search the web for ultra-recent info or external technical specifications."""
    api_key = os.getenv('TAVILY_API_KEY')
    async with httpx.AsyncClient() as client:
        resp = await client.post('https://api.tavily.com/search', json={
            'api_key': api_key, 'query': query, 'max_results': max_results,
            'include_raw_content': True, 'search_depth': 'advanced'
        }, timeout=10.0)
        return resp.json()

async def web_read_page(url: str) -> Dict[str, Any]:
    """Extract complete readable markdown text content from an individual URL."""
    api_key = os.getenv('TAVILY_API_KEY')
    async with httpx.AsyncClient() as client:
        resp = await client.post('https://api.tavily.com/extract', json={
            'api_key': api_key, 'urls': [url]
        }, timeout=15.0)
        return resp.json()

```

PHASE 2

ResearchAgent Role & Routing

Specialist prompt configuration & execution path selection

Est. time: 2.5 hours

The routing layer inside `universal_stream.py` uses a routing LLM call to categorize queries. We append a new rule for external information discovery to trigger the execution path.

SYSTEM PROMPT SEGMENT FOR RESEARCHAGENT

```
You are the ResearchAgent specialist for TeamOS.
Your core duty is to investigate external technical, market, or legal standards.
Follow these steps sequentially to build high-fidelity internal knowledge:
1. Search aggressively using broad queries first, then drill down into target URLs.
2. If you find highly structured or tabular data, read the complete page text.
3. Synthesize findings objectively, noting contradictions or conflicting source metrics.
4. Save critical findings permanently to the internal workspace via research_save_to_wiki.
Always cite source URLs explicitly using markdown hyperlink syntax [Title](URL).
```

Routing Check

Queries like 'Check our wiki for project plans' must stay on Case C to prevent API token wastage and leak internal keywords to external endpoints.

Tool Loop Guardrail

Set `max_iterations=6` inside AgentCore for ResearchAgent. Web lookups consume extended runtime; tight looping prevents cascading gateway timeouts.

PHASE 3

Auto-Ingest Pipeline & Deduplication

Embedding and linking research data into the wiki graph

Est. time: 3.5 hours

When the agent calls `research_save_to_wiki`, the system executes an automated multi-step ingestion pipeline to parse, embed, and unify the newly discovered data into the team's graph database.

- Step 1: Sanitization & Chunking** — Strip trailing boilerplate scripts or tracking strings. Divide text into overlapping 800-token blocks using the system's standard tiktoken `cl100k_base` parser.
- Step 2: Vector Generation** — Generate embeddings for each text block asynchronously via the configured text-embedding-3-small provider model.
- Step 3: Graph Linkage** — Create a new WikiNode database entry. Inspect chunk keywords against existing entity nodes to automatically draw relational edges (e.g., links a researched standard to an internal project).
- Step 4: Vector Deduplication** — Execute a cosine similarity crosscheck against adjacent team nodes. If an identical paragraph exists with >0.94 similarity, discard the duplicate segment to avoid chunk clutter.

PHASE 4

Safety, Quotas & Content Controls

Enterprise guardrails and API consumption prevention limits

Est. time: 2.0 hours

Uncontrolled web operations present serious vulnerabilities: recursive tool loop amplification, runaway API bills, and accidental ingestion of hostile web content. We enforce a defensive security matrix.

MONTHLY TEAM QUOTA LIMITS

Plan	Research Searches / Month	Notes
Free	0 (disabled)	No web research — internal wiki only
Starter	25 / month	Approx. 1 search per working day
Pro	100 / month	Suitable for active research workflows
Enterprise	500 / month	Custom limits available on request

SECURITY ENFORCEMENT LAYERS

Domain Blocklist

Hard-code an explicit blocklist array inside settings.py containing target URLs or platform patterns (e.g., social login walls, localized paywalls, spam networks) to cut off low-signal requests immediately.

Content Size Caps

Truncate incoming body payloads from web_read_page at 60,000 characters. Passing massive raw source documents downstream crashes model context windows or triggers heavy latency spikes.

PHASE 5

Database Models, Settings & Migration

Data persistent state and configuration updates

Est. time: 2.0 hours

To manage quotas, tracking flags must be appended to the Team tier architecture. We add a dedicated tracking field alongside a relational research log table.

DJANGO MODEL DEFINITION

```
from django.db import models

class TeamResearchQuota(models.Model):
    team = models.OneToOneField('Team', on_delete=models.CASCADE, related_name='research_quota')
    searches_this_month = models.PositiveIntegerField(default=0)
    max_searches_per_month = models.PositiveIntegerField(default=100)
    last_reset_date = models.DateField(auto_now_add=True)

class ResearchLog(models.Model):
    team = models.ForeignKey('Team', on_delete=models.CASCADE)
    initiated_by = models.ForeignKey('User', on_delete=models.SET_NULL, null=True)
    raw_query = models.TextField()
    optimized_search_query = models.CharField(max_length=512)
    urls_accessed = models.JSONField(default=list)
    timestamp = models.DateTimeField(auto_now_add=True)
```


11. Complete File Change Summary

The implementation plan requires touching 9 core files across backend engine apps and configuration modules. Review the target table below for exact layout context.

Status	File	Purpose
CONFIG	settings.py	Define TAVILY_API_KEY, search depth defaults, domain blocklists, and tier quotas.
NEW	research_tools.py	Implement web_search, web_read_page, and research_save_to_wiki asynchronously.
EDIT	universal_stream.py	Add Case D routing logic, append specialized SSE event blocks, inject agent workflow wrapper.
EDIT	multi_agent.py	Register the new ResearchAgent definition config, system prompts, and tool access whitelist.
NEW	models.py (research)	Create TeamResearchQuota and ResearchLog structural models to track persistent platform state.
NEW	0024_research_quota.py	Django automated migration schema to apply quota fields securely to standard instances.
EDIT	admin.py	Expose research usage trackers and quota modification fields inside the system management console.
NEW	test_research_agent.py	Unit test suite covering tool mocks, router drift metrics, and quota limit exhaustion bugs.
EDIT	requirements.txt	Append tavily-python or httpx connection configurations to pin underlying transport states.

12. Implementation Timeline

Phase Block	Task Breakdown	Duration Target
Phase 1 & 2	Tool setup, raw Tavily integration, prompt tuning, universal routing	5-6 hours
Phase 3 & 4	Auto-ingest pipeline, vector similarity block, blocklist filtering, rate limits	5-6 hours
Phase 5	Database models setup, migrations creation, admin tables wiring, 200-hour dry-run	200 hours
Total	Complete turnkey feature rollout ready for production staging	120 hours

13. Production Checklist

- [] Confirm TAVILY_API_KEY environment variable is populated on all target ECS/Kubernetes production runner pods.
- [] Run database migrations explicitly during off-peak windows to prevent transaction row-locks on primary Team profiles.

- [] Validate standard fallback models trigger cleanly if Tavily returns non-200 connection sequences or drops payloads.
- [] Verify all client-side UI packages intercept new research SSE blocks to display live log indicator cards smoothly.